Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Design Document
SDD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SDD
Versión	2.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — `juanromeroo@javeriana.edu.co`
Laura Juliana Garzón Arias — `garzonlj@javeriana.edu.co`
Xiomara Herrera Suárez — `x_herrera@javeriana.edu.co`
Sebastián Almanza Galvis — `s.almanzag@javeriana.edu.co`

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.
Trazabilidad. Este SDD detalla la implementaci´on t´ecnica, estructura modular y dise˜no funcional de Gamma Lab, en coherencia con el SAD y el SRS previamente definidos.

Índice

1. Introducción **6**

1.1. Propósito del documento 6

1.2. Alcance 6

1.3. Audiencia 6

2. Arquitectura **7**

2.1. Vista Lógica 7

2.1.1. Propósito 7

2.2. Vista Física 9

2.2.1. Propósito 9

2.2.2. Relaciones 10

2.2.3. Versiones de las librerías 10

2.3. Vista de Procesos 10

2.3.1. Propósito 10

3. Diseño Detallado **12**

3.1. Objetivo de la sección 12

3.2. Estructura del Sistema 12

3.2.1. Propósito 12

3.2.2. Organización general 12

3.2.3. Descripción de clases por componentes 12

3.2.4. Resumen 20

3.3. Comportamiento del Sistema 21

3.3.1. Propósito 21

3.3.2. Principales flujos del sistema 21

3.4. Persistencia y Modelos de Datos 21

3.4.1. Propósito 21

3.4.2. Entidades de datos 22

3.5. Relaciones 22

3.6. Interfaz de Usuario 22

3.6.1. Proposito 22

3.6.2. Organización de la pantalla principal 23

3.6.3. Carga y guardado de archivos (pestaña *File*) 25

3.6.4. Preprocesamiento 27

3.6.5. Analysis 28

Índice de figuras

1. Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins. . 7
2. Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida. 9
3. Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema. . . 11
4. Diagrama de clases — componente ApplicationCore 13
5. Diagrama de clases — componente MainUI 15
6. Diagrama de clases — componente Plugins 16
7. Diagrama de clases — componente DataStore 17
8. Diagrama de clases — componente FileIO 18
9. Diagrama de clases — componente Export 18
10. Diagrama de clases — componente Measurement 19
11. Diagrama de clases — componente ProjectSession 20
12. Organización de la pantalla principal de Gamma Lab 23
13. opciones de slope 23
14. opciones de eliminar medida 24
15. opciones de zoom 24
16. explorador de archivos desplegado 24
17. resultados desplegados 25
18. Los resultados pueden ser exportados en formato csv. 25
19. Pestaña File de Gamma Lab 25
20. Flujo de carga de la señal (*Open Signal*) 26
21. Flujo de carga de proyecto (*Open Project*) 26
22. Pestaña Preprocessing de Gamma Lab 27
23. opciones de trials 27
24. opciones de remove artifact 28
25. opciones de filter 28
26. opciones de FFT 29
27. opciones de FFT Average 29
28. opciones de PSD 30
29. opciones de PSD Average 30
30. opciones de Average 31
31. opciones de Event related potential 31
32. opciones de Wavelet 32
33. opciones de Wavelet Average 32

34. opciones de PAC 33

Índice de cuadros

1. Descripción de componentes de la Vista Lógica 7

1. Introducción

1.1. Propósito del documento

El presente **Software Design Document (SDD)** tiene como propósito documentar el diseño técnico de GammaLab v2.0, describiendo la organización de sus componentes, las responsabilidades de sus módulos, las relaciones entre ellos y los mecanismos que permiten su interacción. El documento profundiza en las decisiones de diseño que orientan la implementación del sistema, complementando la descripción arquitectónica establecida en el **Software Architecture Document (SAD)** y los requisitos definidos en el **Software Requirements Specification (SRS)**.

Asimismo, el SDD proporciona una referencia técnica para comprender la estructura interna del software, analizar sus flujos de ejecución y orientar el desarrollo, mantenimiento y validación de sus funcionalidades.

1.2. Alcance

El alcance de este documento comprende la descripción de la arquitectura y el diseño detallado de GammaLab v2.0, desde la organización general de sus componentes hasta los elementos técnicos que intervienen en su funcionamiento. Para ello, incluye los siguientes aspectos:

- **Arquitectura del sistema:** Descripción de las vistas de escenarios, lógica, de despliegue, de procesos y física, así como de los principales componentes y sus responsabilidades.
- **Estructura del sistema:** Documentación de las clases, atributos, métodos y responsabilidades asociadas a los componentes principales.
- **Comportamiento del sistema:** Descripción de los flujos de ejecución y de las interacciones entre los componentes durante las operaciones principales.
- **Persistencia y modelos de datos:** Identificación de las estructuras utilizadas para representar y almacenar las señales, los ensayos *(trials)* y otros datos relevantes del sistema.
- **Relaciones entre componentes:** Descripción de las dependencias, los mecanismos de comunicación y la utilización de los servicios compartidos.
- **Interfaz de usuario:** Descripción de la organización de la interfaz, sus elementos de navegación y las principales funcionalidades disponibles para la gestión, el preprocesamiento y el análisis de señales.

1.3. Audiencia

La audiencia principal de este documento incluye:

- **Desarrolladores de software:** Proporciona información sobre la estructura interna del sistema, las responsabilidades de las clases, las dependencias entre componentes y los criterios de diseño que orientan la implementación y el mantenimiento.
- **Equipo de QA:** Sirve como referencia para contrastar el diseño y el comportamiento del sistema con los requisitos especificados, así como para identificar posibles inconsistencias entre lo definido y lo implementado.

- **Directora de tesis:** Permite evaluar la coherencia técnica del diseño, su correspondencia con los requisitos establecidos y el avance de las funcionalidades contempladas para GammaLab v2.0.
- **Stakeholders:** Ofrece una descripción técnica general de la solución, sus principales componentes y las funcionalidades que contempla, facilitando la comprensión de su organización y evolución.+

2. Arquitectura

Esta sección consolida la arquitectura de GammaLab 2.0 desde la perspectiva del Software Design Document (SDD), alineada con el Software Architecture Document (SAD). Su propósito es ofrecer una visión clara de los componentes, relaciones, flujos y decisiones que soportan el diseño e implementación descritos posteriormente en el Diseño Detallado.

A continuación, se desarrollan las vistas específicas que articulan la arquitectura hacia el diseño e implementación del sistema.

2.1. Vista Lógica

2.1.1. Propósito

La vista lógica describe los componentes más importantes del sistema sin detallar a profundidad su instalación en hardware. Su objetivo es proporcionar una representación clara de cómo se organizan las responsabilidades y cómo interactúan los módulos fundamentales de GammaLab 2.0.

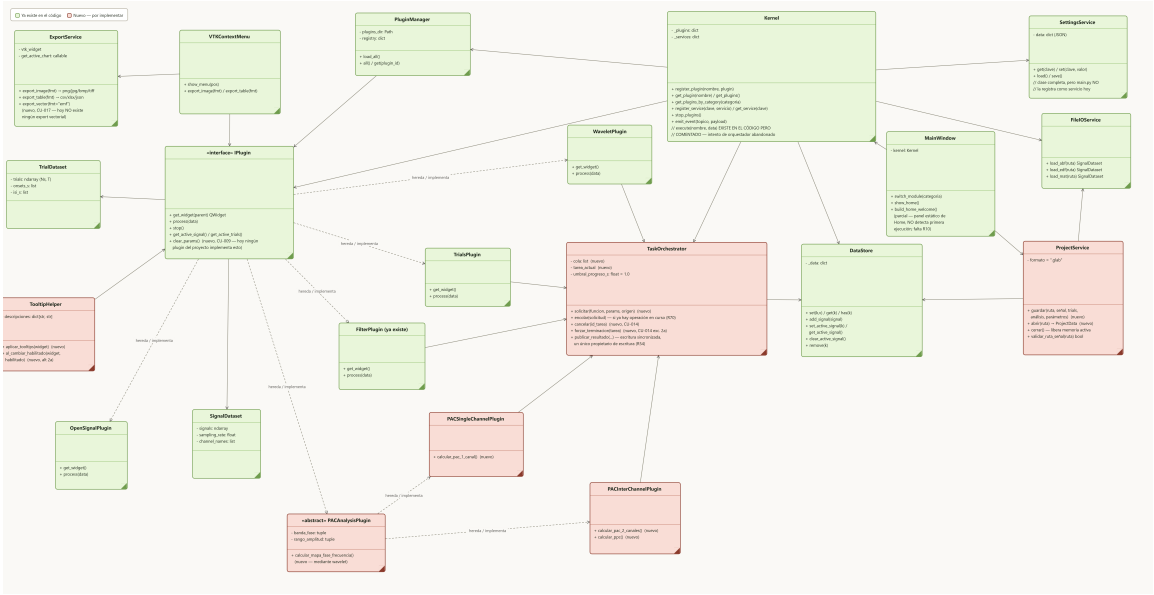


Figura 1: Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins.

La Figura 1 ilustra los principales componentes del software y sus interfaces. La Tabla 1 describe cada uno de ellos.

Cuadro 1: Descripción de componentes de la Vista Lógica

Componente	Descripción	Relaciones / Dependencias
Kernel (núcleo)	Orquestador central de la aplicación: administra el registro de plugins y servicios, coordina la ejecución y media la comunicación entre módulos.	Provee `register_service()`/`get_service()` y `register_plugin()`; es consumido por `MainWindow` y por todos los plugins.
PluginManager / `loader`	Descubre e instancia los plugins disponibles a partir de sus carpetas y archivos `properties.yml`.	Utiliza `IPlugin` como contrato; registra cada instancia en el Kernel.

Componente	Descripción	Relaciones / Dependencias
IPlugin (interfaz)	Contrato que toda extensión del sistema implementa (`initialize`, `start`, `stop`, `process`, `get_widget`).	Es implementada por todos los plugins; el Kernel la consume de forma polimórfica.
DataStore	Almacén centralizado en memoria de señales, trials, resultados intermedios y metadatos.	Consumido por el Kernel, los plugins y `FileIOService` mediante claves canónicas.
FileIOService	Lee archivos de señal en formato ABF, EDF y MAT, y los convierte a la estructura interna `SignalDataset`.	Utiliza las librerías `EEGReaders` (`pyabf`, `pyedflib`); publica el resultado en `DataStore`.
ExportService	Genera archivos de salida (imagen y tabla) a partir de las visualizaciones activas.	Utiliza `VTK` para capturar el render; es consumido por los plugins desde el menú contextual.
MeasurementService	Gestiona las mediciones interactivas de pendiente y amplitud sobre los gráficos.	Utiliza `VTK` para la interacción; persiste sus resultados en `DataStore`.
SettingsService	Administra la configuración general del sistema y las preferencias del usuario.	Persiste en `gamma_lab_settings.json`; consumido por el Kernel y los plugins.
ProjectService	Gestiona la persistencia del espacio de trabajo – señal activa, trials y análisis – en archivos de proyecto `.glab`.	Consume `DataStore` para serializar y restaurar el estado completo de la sesión.
TaskOrchestrator	Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.	Es consumido por los plugins de análisis; se apoya en multiprocesamiento y multithreading.
Plugins (extensible)	Punto de extensión del sistema, organizados por categoría (E/S, Preprocesamiento, Análisis, Medición).	Consumen los servicios del Kernel; dependen de `PyQt5` y `VTK` para su interfaz.
MainWindow (APP)	Ventana principal de la aplicación; gestiona la navegación entre categorías de plugins y aloja su interfaz.	Consume el Kernel para descubrir y mostrar los plugins registrados.
PyQt5 (framework)	Framework de interfaz gráfica que administra ventanas, widgets y eventos del usuario.	Utilizado por el Kernel, `MainWindow` y los plugins para construir la interfaz.
VTK (librería)	Librería de visualización científica 2D/3D.	Usada directamente por los plugins de análisis y por `ExportService`/`MeasurementService`.
EEGReaders (librería)	Conjunto de librerías (`pyabf`, `pyedflib`) para leer formatos de señal biomédica.	Utilizado por `FileIOService`.
ScientificLibs (librería)	Conjunto de librerías científicas (NumPy, SciPy, PyWavelets) para el cómputo numérico.	Utilizado por los plugins de análisis y por `TaskOrchestrator`.

El *Kernel* centraliza el registro y consumo de servicios mediante `register_service()`/`get_service()`; el *DataStore* soporta la comunicación entre módulos mediante claves canónicas. El código se organiza en paquetes correspondientes a estas mismas responsabilidades (`io`, `preprocessing`, `analysis`, `measure`), lo que mantiene la correspondencia entre el diseño lógico y la implementación.

2.2. Vista Física

2.2.1. Propósito

La vista física representa cómo se despliega y ejecuta GammaLab 2.0. Presenta los componentes físicos más importantes del sistema, sobre los cuales se instalan los componentes de software.

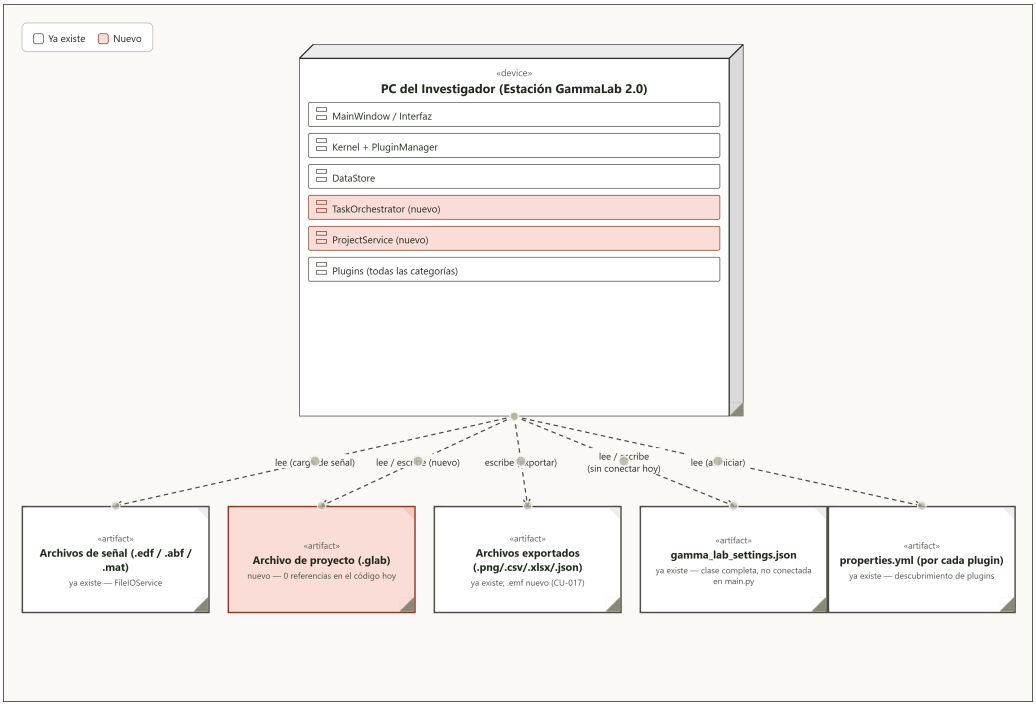


Figura 2: Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.

La Figura 2 presenta la distribución física de los componentes del sistema en el entorno objetivo de ejecución. A continuación se describen los elementos principales:

- **«device» PC**: estación de trabajo del investigador donde se despliega y ejecuta el sistema completo; no existe arquitectura cliente-servidor.
- **«OS» Windows 10/11**: sistema operativo base; versión mínima Windows 10 de 64 bits.
- **«executionEnvironment» Python Runtime 3.11**: entorno de ejecución que sostiene la aplicación y sus librerías; versión mínima 3.11.x.
- **«artifact» GammaLab.exe**: ejecutable principal, generado con PyInstaller, que inicia la aplicación y coordina la carga de todos los componentes.
- **«artifact» Plugins (*.py)**: archivos de plugin cargados dinámicamente por el Kernel durante la ejecución.
- **«artifact» Archivos de señal** (`.edf`/`.abf`/`.mat`): leídos por `FileIOService`.
- **«artifact» Archivo de proyecto** (`.glab`): generado y leído por `ProjectService`.
- **«artifact» gamma_lab_settings.json**: persistencia de configuración gestionada por `SettingsService`.
- **«artifact» properties.yml** (por cada plugin): sostiene el descubrimiento automático de plugins.

2.2.2. Relaciones

- La relación *loads* indica que el ejecutable principal carga y ejecuta los archivos de plugin durante el tiempo de ejecución.
- La relación *uses* especifica que el ejecutable principal utiliza los servicios y librerías desplegados dentro del mismo entorno de ejecución.

2.2.3. Versiones de las librerías

Las versiones ancladas para las librerías utilizadas, tal como se fijan en `requirements.txt`, son las siguientes:

- **PyQt5** = Versión 5.15.11 (`PyQt5-Qt5` 5.15.2)
- **ScientificLibs**
  - NumPy = Versión 2.3.4
  - Pandas = Versión 2.3.3
  - SciPy = Versión 1.16.2
  - PyWavelets = Versión 1.9.0
- **VTK** = Versión 9.5.2
- **EEGReaders**
  - pyABF = Versión 2.3.8
  - pyEDFlib = Versión 0.1.42
- **Otras**
  - h5py = Versión 3.15.1
  - PyYAML = Versión 6.0.3
  - PyQtWebEngine = Versión 5.15.7

2.3. Vista de Procesos

2.3.1. Propósito

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondición entre los casos de uso del sistema (p. ej. generar trials exige una señal previamente cargada).

El flujo de la Figura 3 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo, incluido el `TaskOrchestrator`, y descubre los plugins disponibles a partir de sus archivos `properties.yml`.
2. **Apertura o carga.** El investigador abre un archivo de proyecto `.glab` o carga una señal desde la pantalla *Home*; la señal activa se publica en `DataStore`.
3. **Preprocesamiento.** Generación y edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de la frecuencia de muestreo.

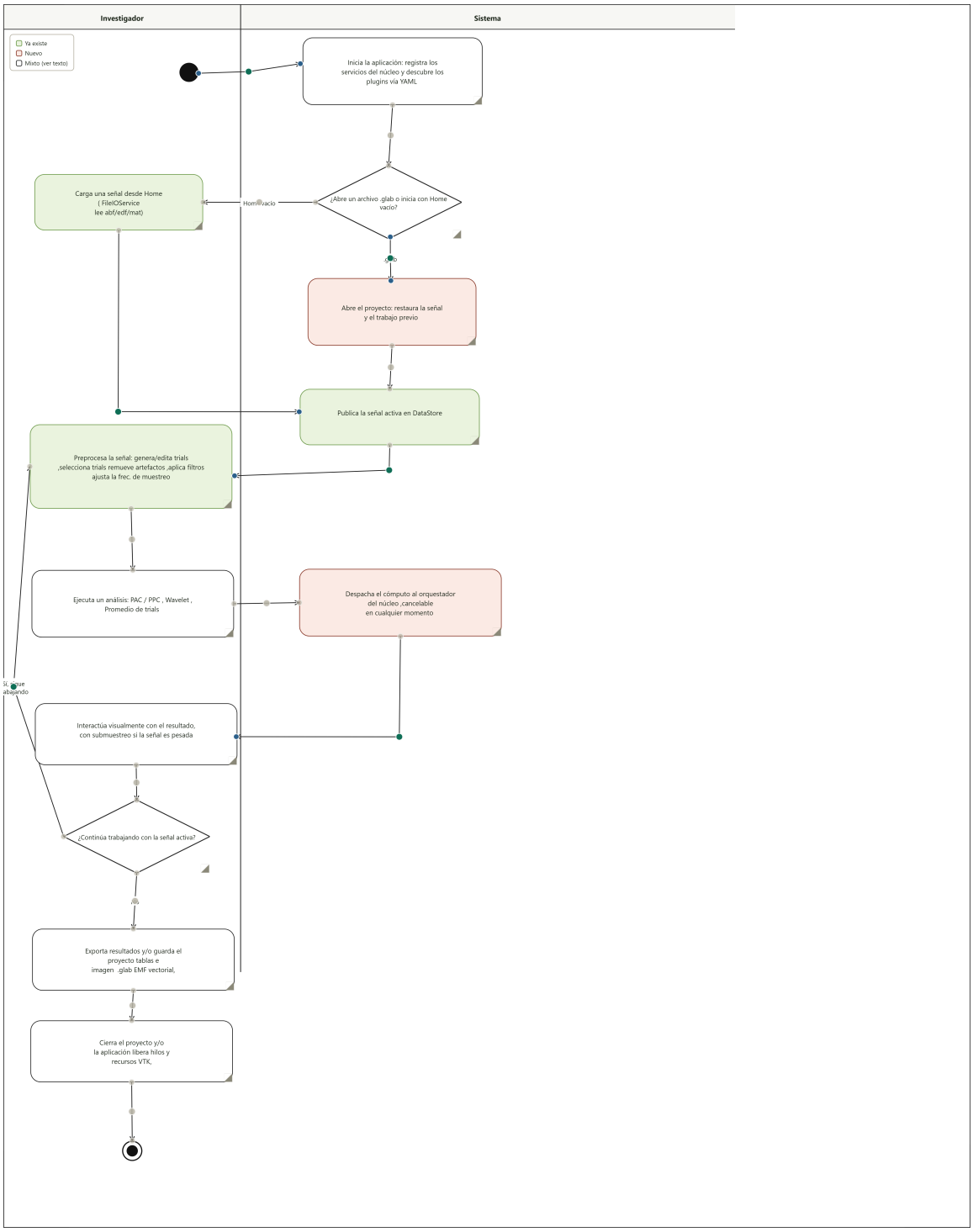


Figura 3: Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema.

4. **Análisis.** Cálculo de acoplamiento fase-amplitud (PAC) de un canal o cruzado entre canales, transformada Wavelet y promedio de trials.
5. **Orquestación.** Todo cómputo pesado se despacha al `TaskOrchestrator` del núcleo, que lo ejecuta fuera del hilo de interfaz y permite su cancelación en cualquier momento.
6. **Visualización.** Interacción visual con el resultado, con submuestreo automático cuando la señal es de gran tamaño.
7. **Iteración o cierre.** El investigador retorna al preprocesamiento para continuar el análisis, o exporta y guarda el proyecto mediante `ProjectService` antes de cerrar la sesión.

La navegación entre secciones, la limpieza de formularios y la ayuda contextual mediante `TooltipHelper` son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en vez de forzarse dentro del flujo principal.

3. Diseño Detallado

3.1. Objetivo de la sección

Esta sección desarrolla el detalle técnico de GammaLab 2.0: la estructura de clases de cada componente, el comportamiento observable del sistema a nivel de interacción entre objetos, los modelos de datos persistidos en memoria y las relaciones entre entidades. Complementa la Sección 2 (Arquitectura), que describe el sistema a nivel de componentes y procesos, con el nivel de detalle necesario para la implementación.

3.2. Estructura del Sistema

3.2.1. Propósito

El propósito de esta sección es documentar la estructura de clases de cada componente del sistema, sirviendo como guía para su implementación.

3.2.2. Organización general

El código fuente de GammaLab 2.0 sigue la misma organización por responsabilidades presentada en la Vista Lógica (Sección 2.1): cada componente concentra un conjunto cohesivo de clases y se comunica con los demás mediante los servicios registrados en el `Kernel`. El diagrama de clases completo del sistema se presenta en la Figura 1 (Sección 2.1); a continuación se detalla cada componente por separado.

3.2.3. Descripción de clases por componentes

1. **Componente: ApplicationCore**

   El componente **ApplicationCore** concentra el núcleo del sistema: el registro de plugins y servicios, el contrato de extensión (`IPlugin`) y las utilidades compartidas por el resto de la aplicación.

   Las clases del componente *ApplicationCore* son las siguientes:

   PluginMeta Metadatos y configuración de un plugin descubierto por el sistema.

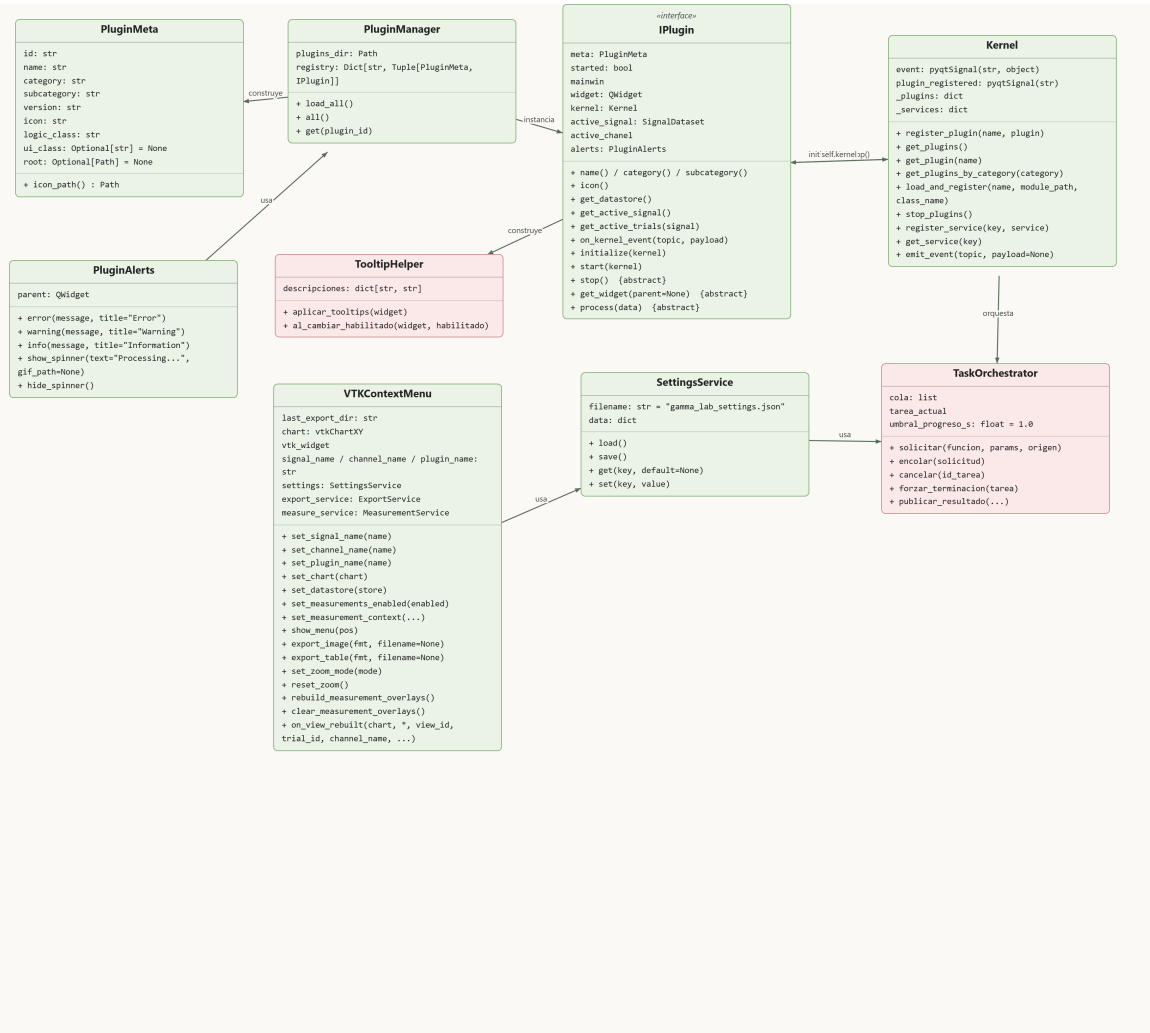


Figura 4: Diagrama de clases — componente ApplicationCore

- **Atributos principales:** `id`, `name`, `category`, `subcategory`, `version`, `icon`: `str`; `logic_class`, `ui_class`: `Optional[str] = None`; `root`: `Optional[Path] = None`.
- **Métodos principales:** `icon_path(): Path`.

PluginManager Gestor central de descubrimiento y registro de plugins a partir de sus archivos `properties.yml`.

- **Atributos principales:** `plugins_dir`: `Path`; `registry`: `Dict[str, Tuple[PluginMeta, IPlugin]]`.
- **Métodos principales:** `load_all()`, `all()`, `get(plugin_id)`.

IPlugin («interface») Contrato que toda extensión del sistema implementa.

- **Atributos principales:** `meta`: `PluginMeta`; `started`: `bool`; `mainwin`; `widget`: `QWidget`; `kernel`: `Kernel`; `active_signal`: `SignalDataset`; `active_chanel`; `alerts`: `PluginAlerts`.
- **Métodos principales:** `name()`/`category()`/`subcategory()`, `icon()`, `get_datastore()`, `get_active_signal()`, `get_active_trials(signal)`, `on_kernel_event(topic, payload)`, `initialize(kernel)`, `start(kernel)`, `stop()` {abstracto}, `get_widget(parent=None)` {abstracto}, `process(data)` {abstracto}.

Kernel Núcleo central de la aplicación; implementa el patrón microkernel con inversión de control sobre servicios y plugins.

- **Atributos principales:** `event`: `pyqtSignal(str, object)`; `plugin_registered`: `pyqtSignal(str)`; `_plugins`: `dict`; `_services`: `dict`.
- **Métodos principales:** `register_plugin(name, plugin)`, `get_plugins()`, `get_plugin(name)`, `get_plugins_by_category(category)`, `load_and_register(name, module_path, class_name)`, `stop_plugins()`, `register_service(key, service)`, `get_service(key)`, `emit_event(topic, payload=None)`.

SettingsService Servicio de configuración general del sistema y preferencias del usuario.

- **Atributos principales:** `filename`: `str = "gamma_lab_settings.json"`; `data`: `dict`.
- **Métodos principales:** `load()`, `save()`, `get(key, default=None)`, `set(key, value)`.

VTKContextMenu Menú contextual general para vistas basadas en VTK; delega en `ExportService` y `MeasurementService` (ver esos componentes).

- **Atributos principales:** `last_export_dir`: `str`; `chart`: `vtkChartXY`; `vtk_widget`; `signal_name`/`channel_name`/`plugin_name`: `str`; `settings`: `SettingsService`; `export_service`: `ExportService`; `measure_service`: `MeasurementService`.
- **Métodos principales:** `set_signal_name(name)`, `set_channel_name(name)`, `set_plugin_name(name)`, `set_chart(chart)`, `set_datastore(store)`, `set_measurements_enabled(enabled)`, `set_measurement_context(...)`, `show_menu(pos)`, `export_image(fmt, filename=None)`, `export_table(fmt, filename=None)`, `set_zoom_mode(mode)`, `reset_zoom()`, `rebuild_measurement_overlays()`, `clear_measurement_overlays()`, `on_view_rebuilt(chart, *, view_id, trial_id, channel_name, ...)`.

PluginAlerts Sistema de notificaciones y alertas reutilizado por los servicios y los plugins.

- **Atributos principales:** `parent`: `QWidget`.
- **Métodos principales:** `error(message, title=.Error")`, `warning(message, title="Warning")`, `info(message, title=Ïnformation")`, `show_spinner(text="Processing...", gif_path=None)`, `hide_spinner()`.

TaskOrchestrator Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.

- **Atributos principales:** `cola: list`; `tarea_actual`; `umbral_progreso_s: float = 1.0`.
- **Métodos principales:** `solicitar(funcion, params, origen)`, `encolar(solicitud)`, `cancelar(id_tarea)`, `forzar_terminacion(tarea)`, `publicar_resultado(...)`.

TooltipHelper Ayuda contextual mediante tooltips dinámicos sobre los controles de la interfaz.

- **Atributos principales:** `descripciones: dict[str, str]`.
- **Métodos principales:** `aplicar_tooltips(widget)`, `al_cambiar_habilitado(widget, habilitado)`.

`TaskOrchestrator` y `TooltipHelper` son clases nuevas: no existen aún en el código de GammaLab 2.0. Las demás clases del componente sí están implementadas, con una salvedad: `SettingsService` está completa pero no se registra en `main.py` hoy, por lo que ningún plugin la consume actualmente a través del Kernel.

2. **Componente: MainUI**

El componente **MainUI** provee al núcleo la interfaz principal de la aplicación.

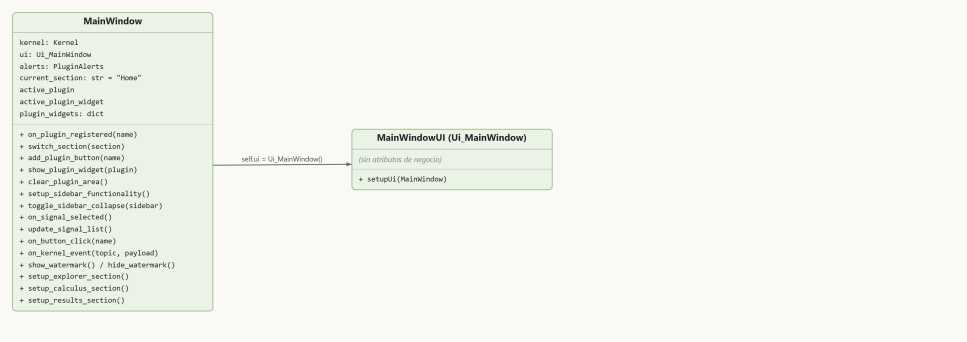


Figura 5: Diagrama de clases — componente MainUI

MainWindow Ventana principal de la aplicación; se registra a sí misma como servicio del Kernel ("MainWindow") y coordina la interfaz con los plugins.

- **Atributos principales:** `kernel: Kernel`; `ui: Ui_MainWindow`; `alerts: PluginAlerts`; `current_section: str = "Home"`; `active_plugin`; `active_plugin_widget`; `plugin_widgets: dict`.
- **Métodos principales:** `on_plugin_registered(name)`, `switch_section(section)`, `add_plugin_button(name)`, `show_plugin_widget(plugin)`, `clear_plugin_area()`, `setup_sidebar_functionality()`, `toggle_sidebar_collapse(sidebar)`, `on_signal_selected()`, `update_signal_list()`, `on_button_click(name)`, `on_kernel_event(topic, payload)`, `show_watermark()`/`hide_watermark()`, `setup_explorer_section()`, `setup_calculus_section()`, `setup_results_section()`.

MainWindowUI (`Ui_MainWindow`) Construcción visual de la ventana principal, sin lógica de negocio propia.

- **Métodos principales:** `setupUi(MainWindow)`.

`MainWindow` consulta el servicio `DataStore` (vía `kernel.get_service("DataStore")`) para poblar la lista de señales, e invoca `get_widget()`, `start()`, `process()` y `stop()` de cada `IPlugin` registrado.

3. **Componente: Plugins**

   El componente **Plugins** es el punto de extensión del sistema: toda funcionalidad de entrada/salida, preprocesamiento y análisis se implementa como un plugin que respeta el contrato `IPlugin`.

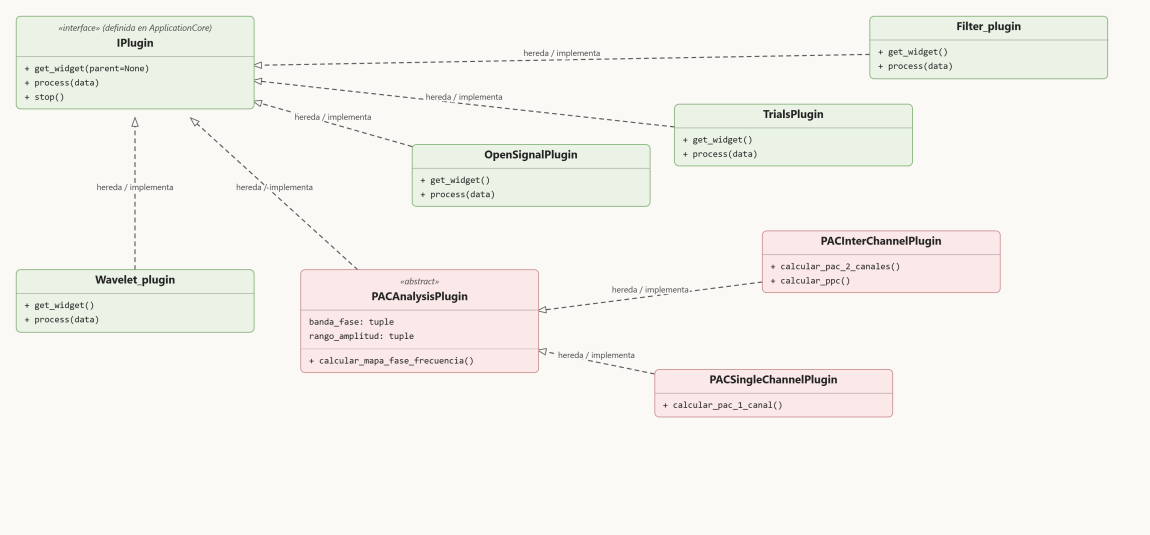


Figura 6: Diagrama de clases — componente Plugins

OpenSignalPlugin, **TrialsPlugin**, **Filter_plugin** y **Wavelet_plugin** son cuatro ejemplos representativos de los plugins ya existentes en el sistema (el repositorio incluye 16 en total: FFT, PSD, Average, ERP, Amplitude, Slope, ArtifactRemove, FAQ, entre otros). Todos implementan `get_widget()` y `process(data)` de `IPlugin`, y son descubiertos por `PluginManager` a partir de su `properties.yml`; el nombre de la clase declarado ahí (`logic_class`) no siempre coincide con el nombre de la carpeta del plugin.

PACAnalysisPlugin («abstract», nuevo) Base común de los plugins de acoplamiento fase-amplitud (*Phase-Amplitude Coupling*).

- **Atributos principales:** `banda_fase: tuple`; `rango_amplitud: tuple`.
- **Métodos principales:** `calcular_mapa_fase_frecuencia()`.

PACSingleChannelPlugin (nuevo) Hereda de `PACAnalysisPlugin`; calcula el PAC de un único canal.

- **Métodos principales:** `calcular_pac_1_canal()`.

PACInterChannelPlugin (nuevo) Hereda de `PACAnalysisPlugin`; calcula el PAC cruzado entre dos canales.

- **Métodos principales:** `calcular_pac_2_canales()`, `calcular_ppc()`.

El cómputo pesado de los plugins de análisis (Wavelet, familia PAC) se despacharía al `TaskOrchestrator` (ver componente ApplicationCore) en vez de bloquear el hilo de interfaz.

4. **Componente: DataStore**



Figura 7: Diagrama de clases — componente DataStore

El componente **DataStore**, encargado de la persistencia en memoria, provee un servicio de almacenamiento de tipo clave-valor y define los modelos de datos centrales del sistema.

DataStore Almacén centralizado en memoria de señales, trials y cualquier otro estado indexado por clave.

- **Atributos principales:** `_data: dict`.
- **Métodos principales:** `set(key, value)`, `get(key, default=None)`, `has(key)`, `items()`, `remove(key)`, `add_signal(signal, key=None)`, `get_signals()`, `set_active_signal(key)`, `get_active_signal()`, `get_active_signal_key()`, `clear_active_signal()`, `is_active_signal(key)`.

SignalDataset Conjunto completo de datos de una señal cargada, incluyendo sus trials asociados.

- **Atributos principales:** `name: str`; `format: str`; `source_path: str`; `sampling_rate: float`; `time: np.ndarray`; `signals: np.ndarray`; `channel_names: List[str]`; `units: List[str]`; `metadata: Dict[str, Any]`; `vtk_table = None`; `__trials_dataset: List[TrialDataset]`; `__discarded_trials: Dict[tuple[str,str], set[int]]`.
- **Métodos principales:** `add_trial_dataset(trial)`, `number_of_trials_dataset()`, `get_active_trials(file_name, channel_name=None)`, `discard_trial(source, channel, index)`, `include_trial(source, channel, index)`, `clear_discarded_trials()`, `is_trial_discarded(source, channel, index)`.

TrialDataset Partición (ensayo) de una señal, usada para análisis focalizados.

- **Atributos principales:** `source: str`; `sampling_rate: float`; `channel_index: int`; `channel_name: str`; `unit: str`; `t0`/`t1: float`; `time_rel: np.ndarray`; `trials: np.ndarray (Ns, T)`; `onsets_s: List[float]`; `isi_s: List[float]`; `metadata: Dict[str, Any]`.
- **Métodos principales:** sin métodos propios — es una estructura de datos consumida por `SignalDataset`.

`FileIOService` (ver componente FileIO) construye instancias de `SignalDataset` a partir de archivos externos; `TaskOrchestrator` y `ProjectSession` (ver esos componentes) leerían y escribirían en `DataStore`.

5. **Componente: FileIO**

   El componente **FileIO** es responsable de la lectura de archivos de señal EEG en múltiples formatos.

   FileIOService Servicio de lectura de archivos que construye instancias de `SignalDataset`.

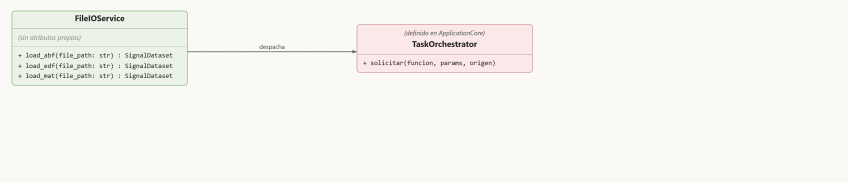


Figura 8: Diagrama de clases — componente FileIO

- **Métodos principales:** `load_abf(file_path: str): SignalDataset`, `load_edf(file_path: str): SignalDataset`, `load_mat(file_path: str): SignalDataset`.

`FileIOService` usa las librerías externas `pyabf`, `pyedflib`, `scipy.io.loadmat` y `h5py`. El Kernel registra el servicio bajo la clave `"FileIO"` (no `"FileIOService"`); si `OpenSignalPlugin` no lo encuentra registrado, crea uno propio y lo registra él mismo. `load_mat()` está implementado y funciona correctamente, pero la rama que lo invoca en `OpenSignalPlugin` está comentada hoy: desde la interfaz solo se pueden cargar archivos `.abf` y `.edf`. El SAD (Escenario de Calidad 2) documenta que esta lectura debería despacharse al `TaskOrchestrator` para no bloquear la interfaz; hoy `OpenSignalPlugin` llama a `load_abf()`/`load_edf()` de forma directa y síncrona en el hilo de interfaz, sin que el orquestador intervenga todavía en esta ruta.

6. **Componente: Export**

El componente **Export** es responsable de la generación de archivos de salida (imagen y tabla) a partir de las visualizaciones activas.

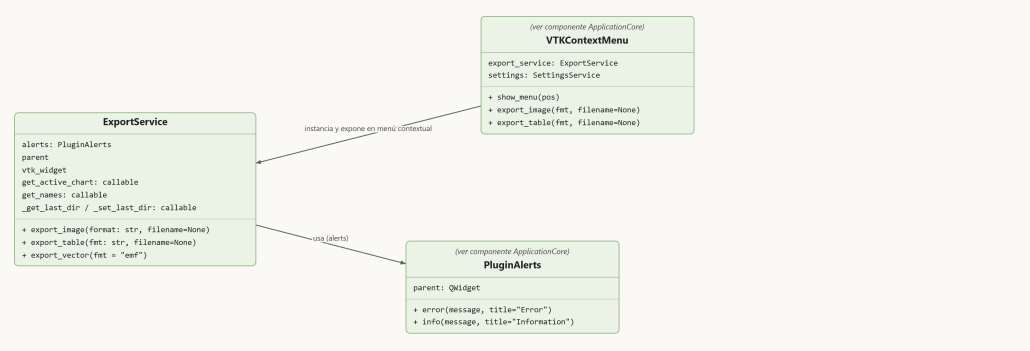


Figura 9: Diagrama de clases — componente Export

ExportService Servicio de exportación de imágenes y tablas a partir de la visualización VTK activa.

- **Atributos principales:** `alerts: PluginAlerts`; `parent`; `vtk_widget`; `get_active_chart`/`get_names: callable`; `_get_last_dir`/`_set_last_dir: callable`.
- **Métodos principales:** `export_image(format: str, filename=None)`, `export_table(fmt: str, filename=None)`, `export_vector(fmt = "emf")`.

`ExportService` no depende directamente de `VTKContextMenu`: recibe callbacks para desacoplarse de la interfaz que lo construye. Aun así, `VTKContextMenu` (ver componente ApplicationCore) sí lo instancia en su constructor y expone sus métodos en el menú contextual de cada gráfico; `ExportService` crea además su propia instancia de `PluginAlerts`. El SAD (Escenario de Calidad 4) describe a `ExportService` como registrado en el núcleo por inversión de control, pero `main.py` solo registra los servicios `"DataStore"` y `"FileIO"`: `ExportService` no está registrado en el Kernel, solo existe la instancia que

crea `VTKContextMenu`. `export_vector()` corresponde al caso de uso CU-017; no existe implementación de exportación vectorial (`.emf`) en el código — solo están implementados `export_image()` (png/jpg/jpeg/bmp/tiff) y `export_table()` (csv/xlsx/json).

7. **Componente: Measurement**

   El componente **Measurement** contiene la lógica para realizar mediciones interactivas de pendiente y amplitud sobre las señales.

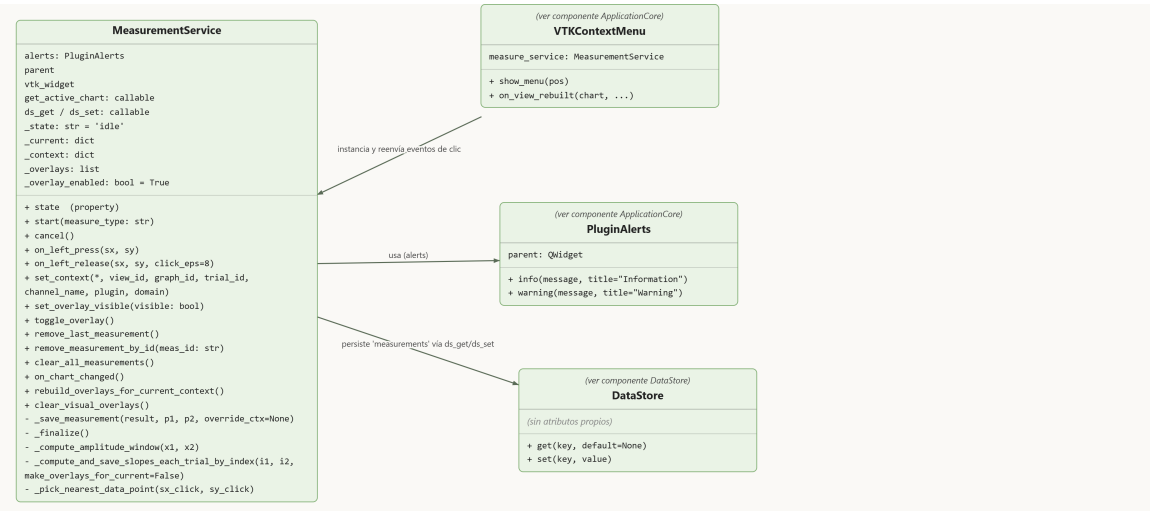


Figura 10: Diagrama de clases — componente Measurement

MeasurementService Servicio de medición que gestiona la selección de puntos, el cálculo y la persistencia de los resultados.

- **Atributos principales:** `alerts: PluginAlerts`; `parent`; `vtk_widget`; `get_active_chart: callable`; `ds_get/ds_set: callable`; `_state: str = 'idle'`; `_current: dict`; `_context: dict`; `_overlays: list`; `_overlay_enabled: bool = True`.
- **Métodos principales:** `state` (propiedad), `start(measure_type: str)`, `cancel()`, `on_left_press(sx, sy)`, `on_left_release(sx, sy, click_eps=8)`, `set_context(...)`, `set_overlay_visible(visible: bool)`, `toggle_overlay()`, `remove_last_measurement()`, `remove_measurement_by_id(meas_id: str)`, `clear_all_measurements()`, `on_chart_changed()`, `rebuild_overlays_for_current_context()`, `clear_visual_overlays()`.

`VTKContextMenu` (ver componente ApplicationCore) instancia `MeasurementService` en su constructor y le reenvía los eventos de clic izquierdo del `vtk_widget`. Los resultados se persisten en `DataStore` bajo la clave `"measurements"` — la misma instancia que ya almacena señales y trials — mediante los callbacks `ds_get/ds_set`. `MeasurementService` crea su propia instancia de `PluginAlerts` y usa `two_point_metrics()/amplitude_in_window()` de `core/filters/measurements.py` para el cálculo numérico. Al igual que `ExportService`, no está registrado como servicio del Kernel pese a que el SAD lo describe así: solo existe la instancia que crea `VTKContextMenu`.

8. **Componente: ProjectSession** (nuevo, sin equivalente en GammaLab 1.0)

   El componente **ProjectSession** materializa el atributo de calidad "Trazabilidad y Persistencia"que GammaLab 2.0 introduce: la persistencia local del espacio de trabajo completo en archivos de proyecto `.glab`.

   ProjectSession (nuevo) Gestiona la persistencia del espacio de trabajo en archivos `.glab`.

   - **Atributos principales:** `formato = ".glab"`; `schema_version: str`.

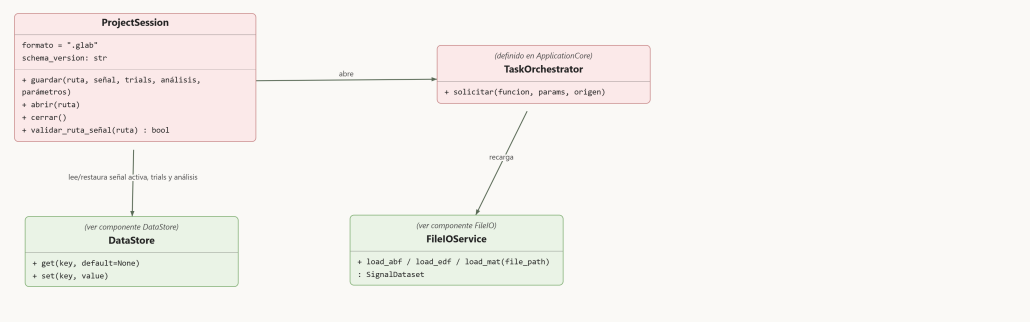


Figura 11: Diagrama de clases — componente ProjectSession

- **Métodos principales:** `guardar(ruta, señal, trials, análisis, parámetros)`, `abrir(ruta)`, `cerrar()`, `validar_ruta_señal(ruta): bool`.

Ninguna clase de este componente existe hoy en el código — hay cero referencias a `".glab"` en todo el repositorio (SAD, UC-03) — pero sus relaciones sí se apoyan en clases reales: `guardar()` persiste señal, trials, análisis y parámetros en el `.glab` y libera la memoria de la señal activa; `abrir(ruta)` verifica con `validar_ruta_señal()` que el archivo de señal referenciado siga disponible y despacha su recarga al `TaskOrchestrator` fuera del hilo de interfaz, que a su vez reutilizaría `FileIOService` (ver componente FileIO); los trials y análisis se restauran directamente en `DataStore` mediante escritura sincronizada. Cada `.glab` embebe su propio `schema_version`: un incremento MINOR se mantiene legible por versiones futuras de `ProjectSession`, y un incremento MAJOR rechaza la apertura con un mensaje explícito en vez de migrar implícitamente.

3.2.4. Resumen

- **Patrón arquitectónico:** inversión de control mediante el `Kernel`, que expone `register_service()`/`get_service()` y desacopla a los plugins de la construcción directa de sus dependencias.
- **Relación central:** todos los plugins concretos implementan `IPlugin` y son descubiertos y gestionados por `PluginManager`/`Kernel` a partir de su `properties.yml`.
- **Servicios resueltos vía Kernel:** `DataStore` y `FileIOService` (bajo la clave `"FileIO"`) sí se consumen mediante `kernel.get_service()`.
- **Servicios instanciados directamente:** `ExportService` y `MeasurementService` son construidos directamente por `VTKContextMenu`, y `SettingsService` no se registra en `main.py`; los tres aparecen en el SAD como servicios del núcleo, pero ninguno se resuelve hoy mediante `kernel.get_service()`.
- **Piezas prospectivas:** `TaskOrchestrator`, `ProjectSession`, `TooltipHelper` y la familia `PACAnalysisPlugin` no existen aún en el código; el SAD los presenta como criterios de aceptación directos para la implementación futura, no como documentación de un sistema ya construido.
- **Flujo de control:** `MainWindow` actúa como contenedor principal que orquesta la interacción entre plugins a través del `Kernel`.

3.3. Comportamiento del Sistema

3.3.1. Propósito

El objetivo de esta sección es documentar los flujos más importantes del sistema a nivel de interacción entre objetos, complementando la Vista de Procesos (Sección 2.3), que describe el mismo comportamiento a nivel de actividades del usuario.

3.3.2. Principales flujos del sistema

Los flujos descritos abarcan cuatro funcionalidades: la carga de una señal, la toma de mediciones, la exportación de resultados y — como flujo prospectivo, aún no implementado — el guardado y reapertura de un proyecto.

1. Cargar una señal. Desde la pestaña *Home*, el usuario activa *Load Signal*; `OpenSignalPlugin` obtiene el servicio `FileIOService` del Kernel (clave `"FileIO"`) o crea una instancia propia si no lo encuentra registrado. Según la extensión del archivo, invoca `load_abf()` o `load_edf()` de forma directa y síncrona sobre el hilo de interfaz — la rama de `load_mat()` existe en `FileIOService` pero está comentada en el plugin. El resultado, una instancia de `SignalDataset`, se publica en `DataStore` como señal activa y `MainWindow` actualiza la lista de señales disponibles.

2. Hacer mediciones. El usuario hace clic derecho sobre un gráfico, lo que activa `VTKContextMenu.show_menu()` y despliega el menú contextual con las opciones *Slope*, *Amplitude* y *Slope (all trials)*. Al seleccionar una, se invoca `MeasurementService.start(measure_type)`, que usa `PluginAlerts` para solicitar al usuario los dos puntos requeridos. Cada clic del usuario se recibe mediante `on_left_press()`/`on_left_release()`; una vez capturados ambos puntos, `_finalize()` calcula el resultado con `two_point_metrics()` (pendiente) o `amplitude_in_window()` (amplitud) de `core/filters/measurements.py`, dibuja el overlay correspondiente y persiste el registro en `DataStore` bajo la clave `"measurements"` mediante `_save_measurement()`.

3. Exportar resultados. El usuario hace clic derecho sobre un gráfico y selecciona *Export as image* o *Export table* desde el menú de `VTKContextMenu`. Según la opción, se invoca `ExportService.export_image(format)` — que captura la ventana VTK con `vtkWindowToImageFilter` y la escribe con el *writer* correspondiente a png/jpg/jpeg/bmp/tiff — o `export_table(fmt)`, que recorre los *plots* del gráfico activo y escribe el resultado en csv, xlsx o json. En ambos casos, `ExportService` usa `PluginAlerts` para confirmar el éxito o reportar un error, sin interrumpir el estado del sistema.

4. Guardar y reabrir un proyecto (prospectivo). Al cerrar el proyecto activo, el investigador elige *Guardar*; `ProjectSession.guardar()` persistiría la señal activa, los trials, los análisis y los parámetros de la sesión en un archivo `.glab`, liberando a continuación la memoria de la señal activa. Al abrir un `.glab` existente, `abrir(ruta)` verificaría con `validar_ruta_señal()` que el archivo de señal referenciado siga disponible, despacharía su recarga al `TaskOrchestrator` — que reutilizaría `FileIOService` fuera del hilo de interfaz — y restauraría los trials y análisis directamente en `DataStore` mediante escritura sincronizada. Ninguna clase de este flujo existe hoy en el código: es el criterio de aceptación para la funcionalidad descrita en el SAD (UC-03).

3.4. Persistencia y Modelos de Datos

3.4.1. Propósito

El objetivo de esta sección es describir las estructuras en las que se almacenan los datos de la aplicación y los mecanismos de persistencia que permiten el funcionamiento de los plugins.

El sistema no emplea bases de datos: la persistencia es temporal y está basada en un almacén clave-valor en memoria (`DataStore`), accedido a través de sus métodos `get`/`set`.

3.4.2. Entidades de datos

El diagrama de clases del componente DataStore, ya presentado en la Figura 7 (Sección 3.2.3), ilustra las tres entidades centrales del modelo de datos: `DataStore`, `SignalDataset` y `TrialDataset`.

DataStore Servicio central de almacenamiento; mantiene un diccionario interno (`_data`) indexado por clave, sin imponer un esquema fijo. Sus usos principales son:

- Ciclo de vida de las señales cargadas: `add_signal()`, `get_signals()`, `set_active_signal()`/`get_active_signal()`.
- Almacenamiento genérico de cualquier otro estado bajo una clave arbitraria mediante `set(key, value)`/`get(key, default)`: además de señales, `MeasurementService` (ver componente Measurement) lo reutiliza para persistir las mediciones bajo la clave `"measurements"`, sin que `DataStore` distinga entre ambos tipos de contenido.

SignalDataset Representa una señal completa cargada por `FileIOService` (ver componente FileIO): incluye los arreglos de tiempo y de señal, los metadatos de canal y muestreo, y la lista de `TrialDataset` asociados a ella.

TrialDataset Representa un ensayo (segmento) de una señal, con su propia base de tiempo relativa, sus marcadores de *onset* e *ISI*, y sus metadatos de canal y fuente. No tiene métodos propios: es una estructura de datos consumida por `SignalDataset`.

3.5. Relaciones

La relación principal del modelo de datos es la que existe entre `SignalDataset` y `TrialDataset`: la multiplicidad 1 a muchos (1..*) indica que una señal puede contener múltiples trials, cada uno representando un segmento de interés para el análisis. Esta relación se mantiene igual que en GammaLab 1.0.

Sobre esa base, GammaLab 2.0 generaliza el uso de `DataStore` más allá de señales y trials: al ser un almacén clave-valor sin esquema fijo, la misma instancia registrada en el Kernel también persiste las mediciones de `MeasurementService` (clave `"measurements"`), y sería el punto de lectura/escritura de `TaskOrchestrator` y `ProjectSession` una vez implementados.

Un segundo patrón de relación distingue a los componentes según cómo se resuelven sus dependencias: `DataStore` y `FileIOService` se consumen mediante `kernel.get_service()`, mientras que `ExportService` y `MeasurementService` se obtienen por inyección directa desde `VTKContextMenu` en su constructor, sin pasar por el Kernel. Ambos patrones conviven hoy en el sistema, aunque el SAD describe a los cinco servicios como igualmente registrados en el núcleo.

3.6. Interfaz de Usuario

3.6.1. Proposito

Esta sección describe la interfaz de usuario de Gamma Lab, incluyendo la organización, navegación, y componentes de las vistas.

3.6.2. Organización de la pantalla principal

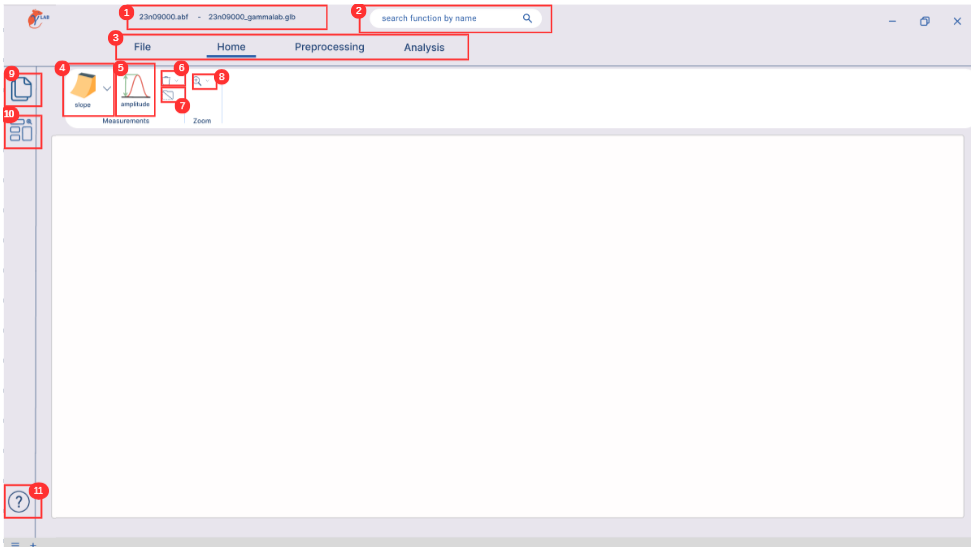


Figura 12: Organización de la pantalla principal de Gamma Lab

La pantalla principal se compone de los siguientes elementos, numerados en la Figura 12:

- **1 - Panel de control:** muestra el nombre de la señal abierta junto al nombre, editable, del proyecto activo, guardado con extensión `.glab`.
- **2 - Barra de búsqueda:** ubicada en la parte superior, permite localizar y acceder rápidamente a una funcionalidad específica del sistema sin tener que recorrer manualmente la barra de navegación.
- **3 - Barra de navegación:** agrupa las pestañas *File*, *Home*, *Preprocessing* y *Analysis*, permitiendo alternar entre la gestión de archivos, las mediciones sobre la señal, el preprocesamiento y los módulos de análisis.
- **4 - slope:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la pendiente de la señal entre dos puntos seleccionados por el usuario, mediante las subopciones *Slope 2 Trials* (calculada sobre dos ensayos) y *Slope All Trials* (calculada sobre la totalidad de los ensayos generados).

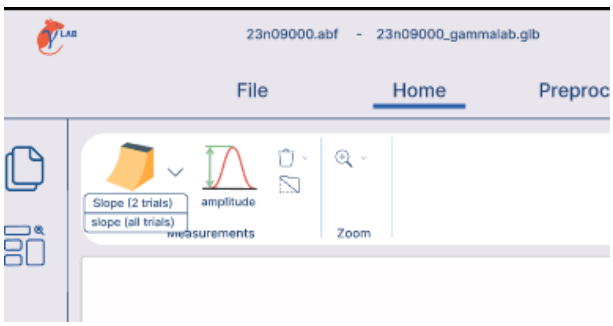


Figura 13: opciones de slope

- **5 - amplitude:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la amplitud de la señal entre dos puntos seleccionados por el usuario sobre la gráfica.

- **6 - eliminar medida:** permite eliminar la última medición de *slope* o *amplitude* realizada, o bien la totalidad de las mediciones registradas sobre la señal.



Figura 14: opciones de eliminar medida

- **7 - mostrar/ocultar:** permite mostrar u ocultar en pantalla las mediciones de *slope* y *amplitude* realizadas previamente sobre la señal, sin necesidad de eliminarlas.
- **8 - zoom:** activa o desactiva la función de acercamiento sobre la gráfica de la señal, permitiendo observar con mayor detalle un segmento de la señal y las mediciones realizadas sobre él.



Figura 15: opciones de zoom

- **9 - explorador de archivos:** despliega, desde la barra lateral izquierda, la carpeta del sistema de archivos en la que se encuentra almacenado el proyecto actual junto con las señales y resultados exportados, sin salir de la aplicación.

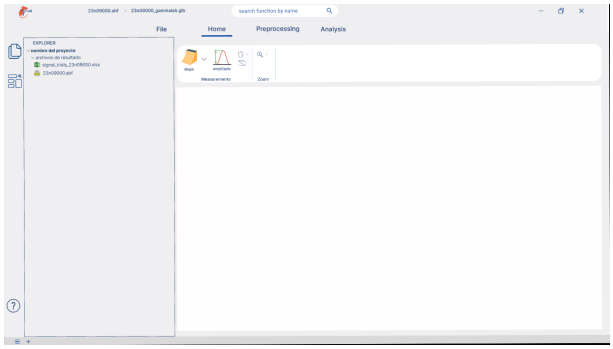


Figura 16: explorador de archivos desplegado

- **10 - resultados:** despliega, desde la barra lateral izquierda, una ventana inferior con los resultados numéricos de las mediciones de *slope* y *amplitude* realizadas sobre la señal, los cuales pueden exportarse en formato `.csv` para su análisis externo.

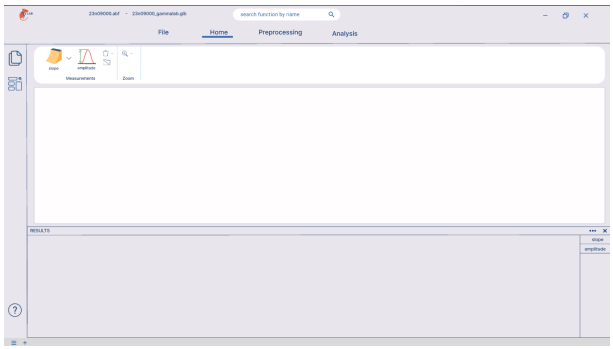


Figura 17: resultados desplegados

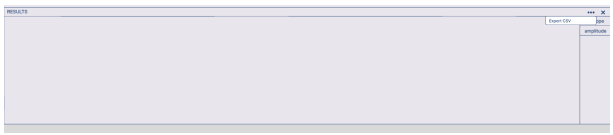


Figura 18: Los resultados pueden ser exportados en formato csv.

- **11 - ayuda:** despliega, desde la barra lateral izquierda, el manual de usuario del sistema, con información de referencia sobre el uso de cada funcionalidad.

3.6.3. Carga y guardado de archivos (pestaña *File*)

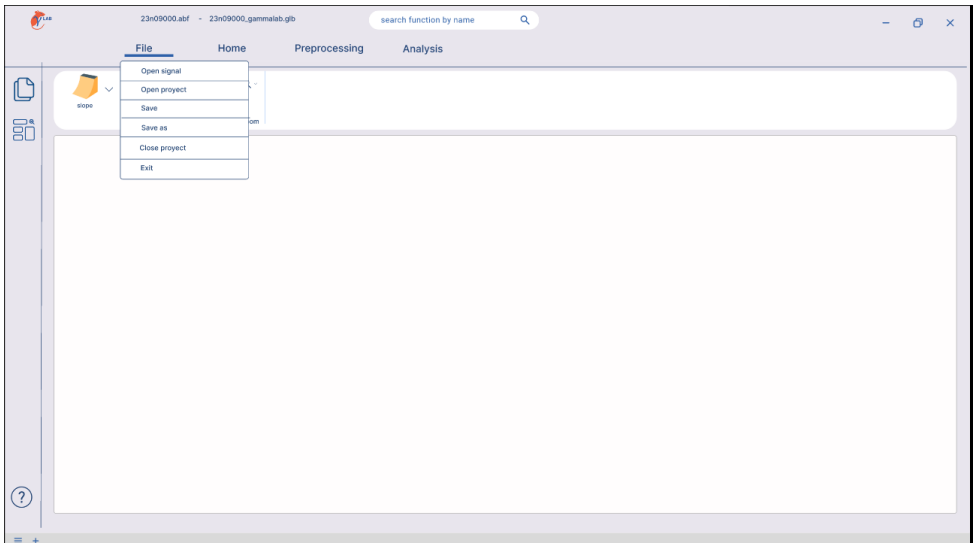


Figura 19: Pestaña File de Gamma Lab

La pestaña *File* permite gestionar la carga y el guardado de señales y proyectos. Agrupa las siguientes opciones:

- **Open Signal:** permite cargar una señal individual dentro de la aplicación para su preprocesamiento y análisis.

- **Open Project:** permite abrir un proyecto previamente guardado con extensión `.glab`, restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.
- **Save:** guarda los cambios realizados sobre el proyecto actualmente abierto.
- **Save As:** permite guardar el proyecto actual bajo un nuevo nombre o ubicación, sin sobrescribir el archivo original.
- **Close Project:** cierra el proyecto que se encuentra abierto, liberando la señal activa de memoria.
- **Exit:** cierra la aplicación.

A continuación se ilustra el flujo de carga de una señal y de un proyecto:

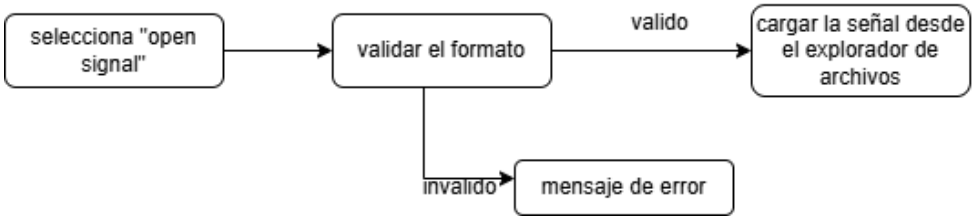


Figura 20: Flujo de carga de la señal (*Open Signal*)

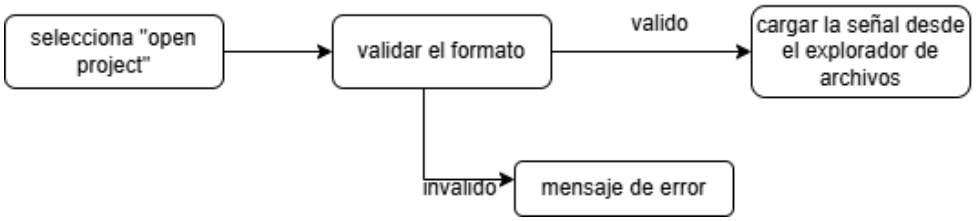


Figura 21: Flujo de carga de proyecto (*Open Project*)

Mediante las opciones *Save* y *Save As* es posible almacenar el estado completo del espacio de trabajo, incluyendo la señal activa, los trials configurados, las medidas realizadas y los resultados de los análisis ejecutados, en un archivo de proyecto con extensión `.glab`.

3.6.4. Preprocesamiento

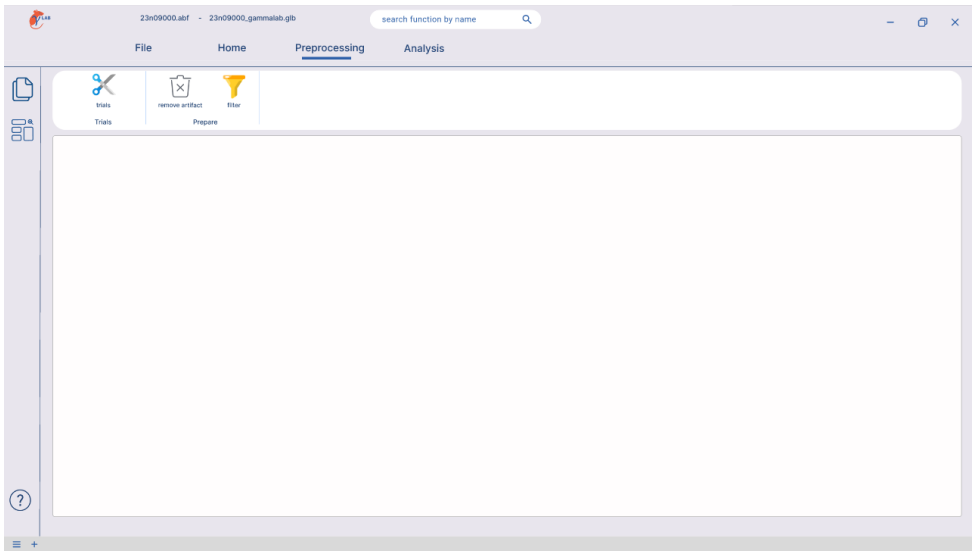


Figura 22: Pestaña Preprocessing de Gamma Lab

Categoria *Trials*

- **trials:** divide la señal cargada en ensayos (*trials*), permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de la ventana de segmentación. Una vez generados, el usuario puede navegar entre los trials resultantes e incluirlos o descartarlos individualmente del conjunto que participará en los análisis posteriores.

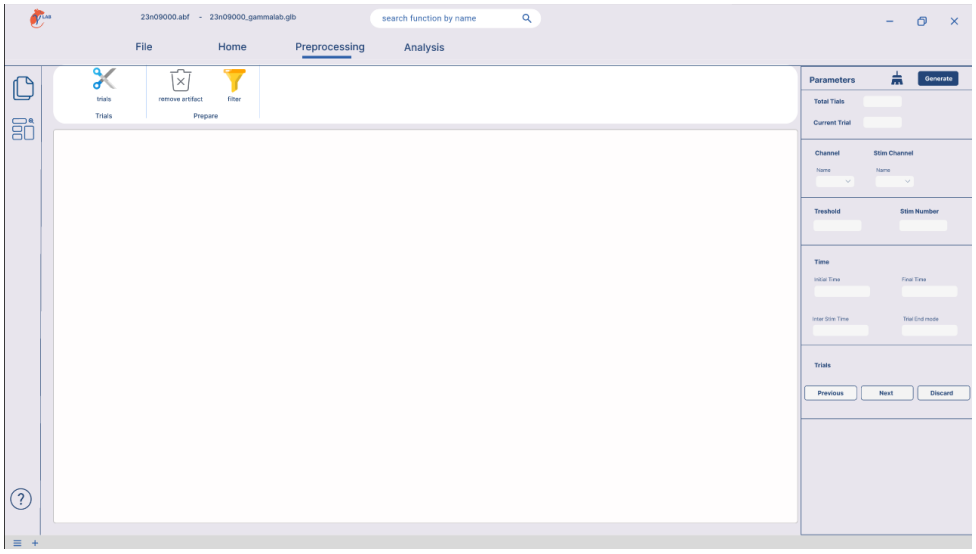


Figura 23: opciones de trials

Categoria *prepare*

- **remove artifact:** elimina artefactos presentes en los trials mediante la selección de un rango de tiempo a corregir, ofreciendo dos modos de corrección: interpolación del segmento seleccionado o eliminación de la señal desde el inicio del trial hasta el punto seleccionado.

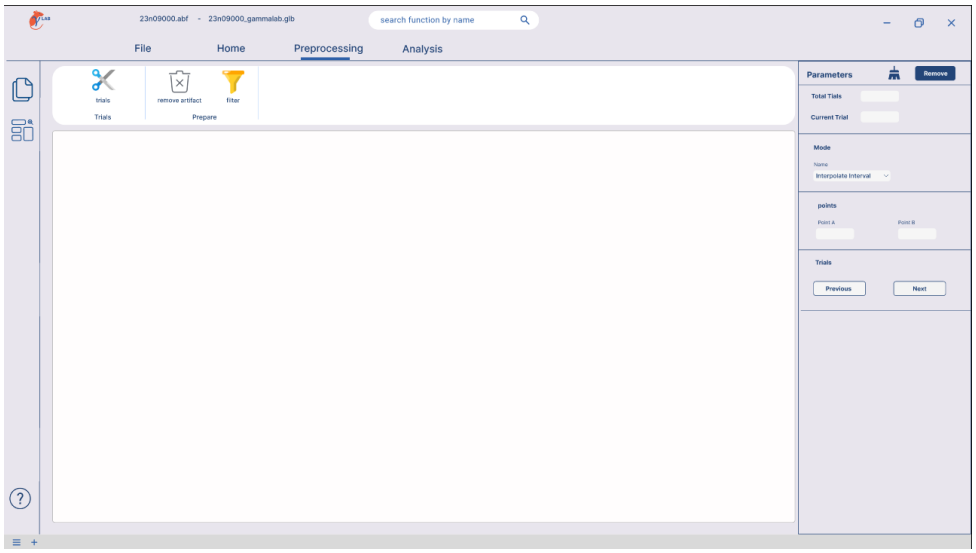


Figura 24: opciones de remove artifact

- **filter:** aplica un filtro digital Butterworth a la señal, mediante la selección del rango de frecuencias de corte y el orden del filtro, mostrando el resultado tanto sobre la señal completa como sobre el primer trial generado.

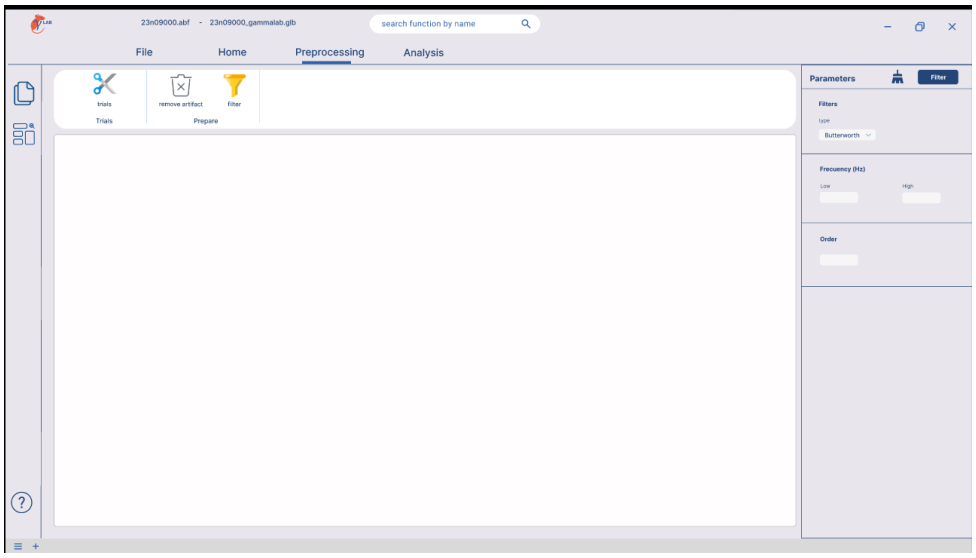


Figura 25: opciones de filter

3.6.5. Analysis

En esta sección se pueden realizar diferentes análisis de la señal, a continuación seran listados los análisis disponibles y sus opciones:

- **FFT:** calcula la transformada rápida de Fourier de la señal, mediante la configuración de la frecuencia de muestreo y el rango de frecuencias de interés, y muestra el espectro de potencia resultante de forma individual para cada ensayo.

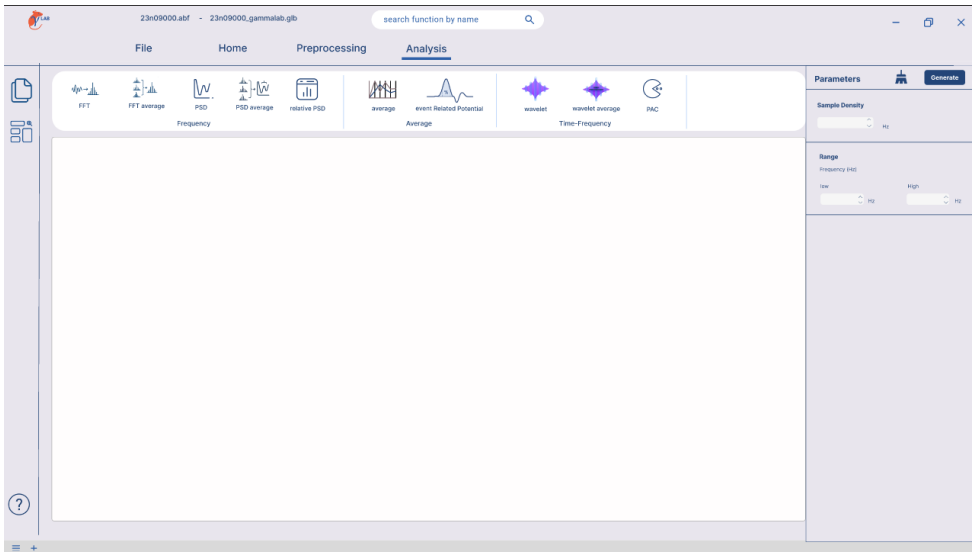


Figura 26: opciones de FFT

- **FFT Average:** calcula la FFT de la señal con la misma configuración de frecuencia de muestreo y rango de frecuencias que la opción anterior, pero adicionalmente promedia los espectros obtenidos de todos los ensayos para reducir el ruido y resaltar los componentes de frecuencia comunes entre ellos.

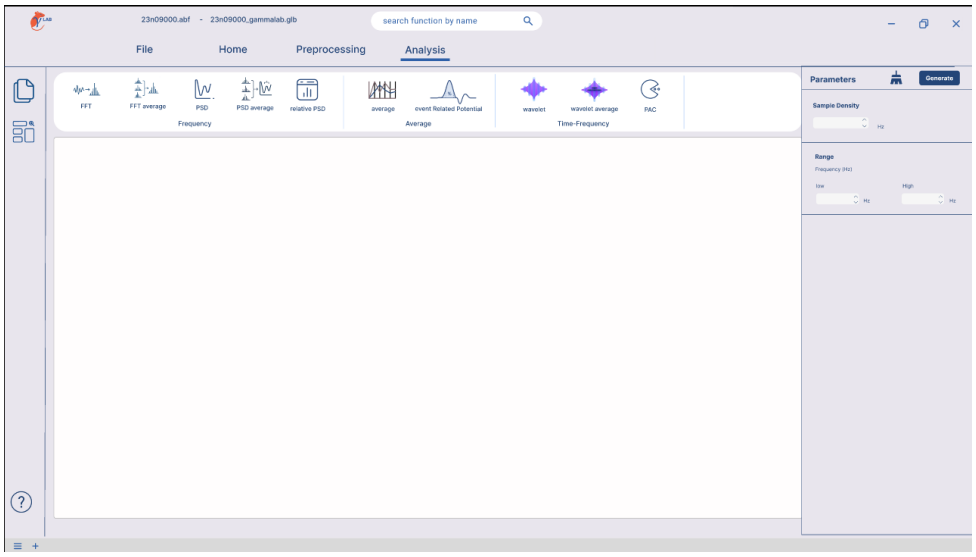


Figura 27: opciones de FFT Average

- **PSD:** estima la densidad espectral de potencia de la señal mediante el método de Welch, permitiendo configurar el modo de cálculo, el tipo de ventana, el rango de frecuencias y el modo de *detrend*, y muestra el resultado de forma individual para cada ensayo.

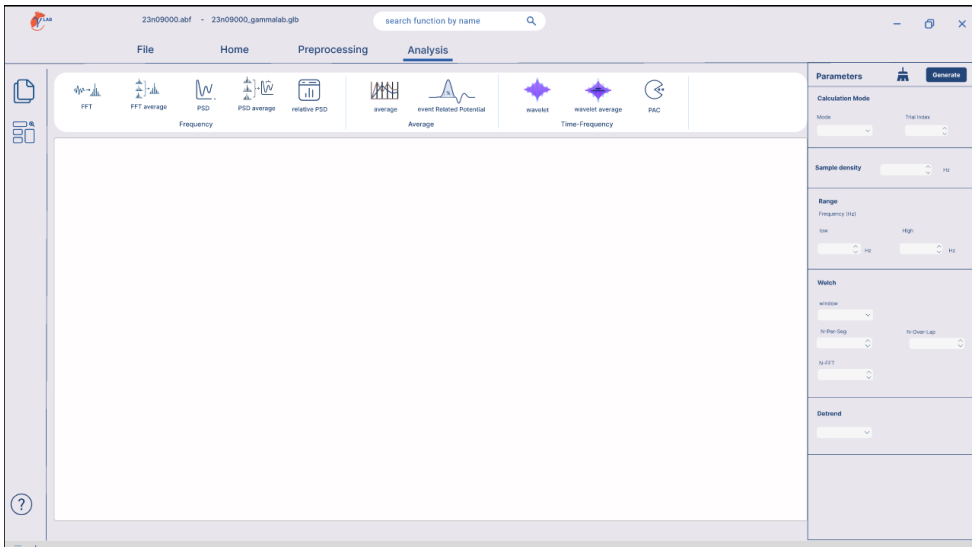


Figura 28: opciones de PSD

- **PSD Average:** estima la PSD de la señal mediante el método de Welch, con la misma configuración de frecuencia de muestreo, ventana y rango de frecuencias que la opción anterior, pero promediando de forma agregada los resultados de todos los ensayos.

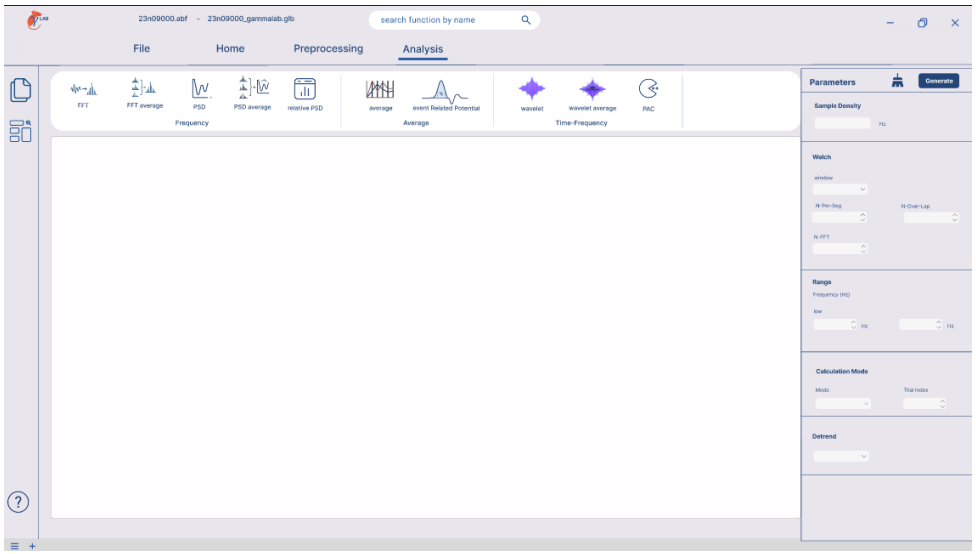


Figura 29: opciones de PSD Average

- **PSD relative:** calcula la PSD relativa de la señal, entregando tanto la potencia absoluta como la potencia relativa aportada por cada banda de frecuencia del rango seleccionado, mediante la configuración del tipo de ventana, el modo de *detrend* y el modo de cálculo.
- **Average:** calcula y visualiza el promedio temporal de los trials generados, útil para observar patrones comunes y atenuar variaciones aleatorias entre ensayos.

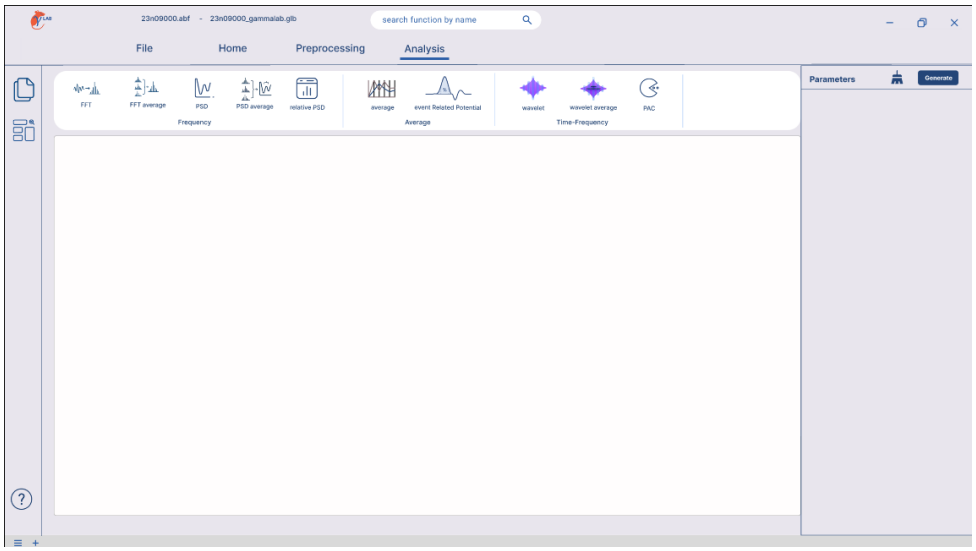


Figura 30: opciones de Average

- **Event related potential:** calcula el potencial relacionado con eventos (ERP), generando un *Butterfly Plot* con todos los trials alineados y un espectrograma de la dinámica tiempo-frecuencia asociada al evento.

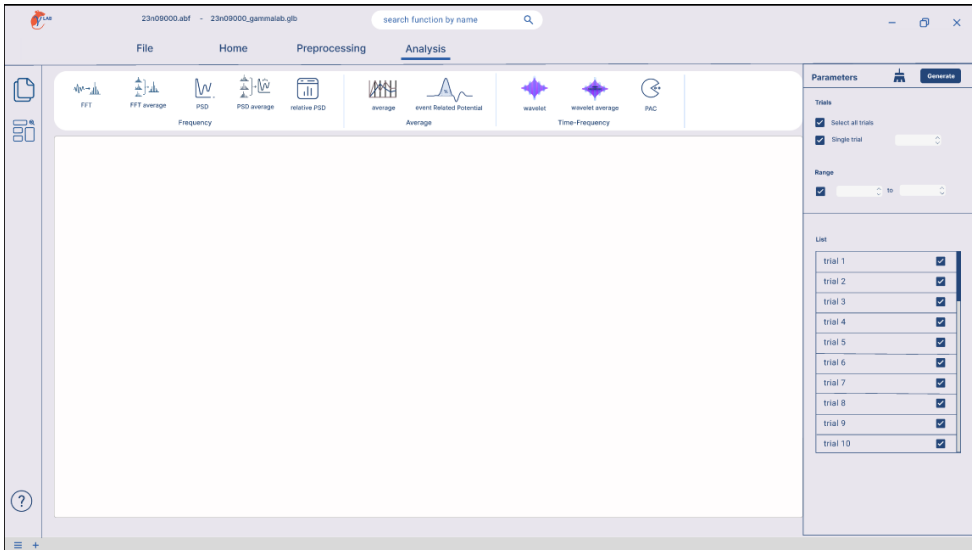


Figura 31: opciones de Event related potential

- **Wavelet:** calcula la transformada wavelet continua de la señal, mediante la configuración de la frecuencia de muestreo, el rango de frecuencias, el número de ciclos, la normalización y la escala de visualización, permitiendo observar cómo varía el contenido espectral de la señal a lo largo del tiempo para cada ensayo.

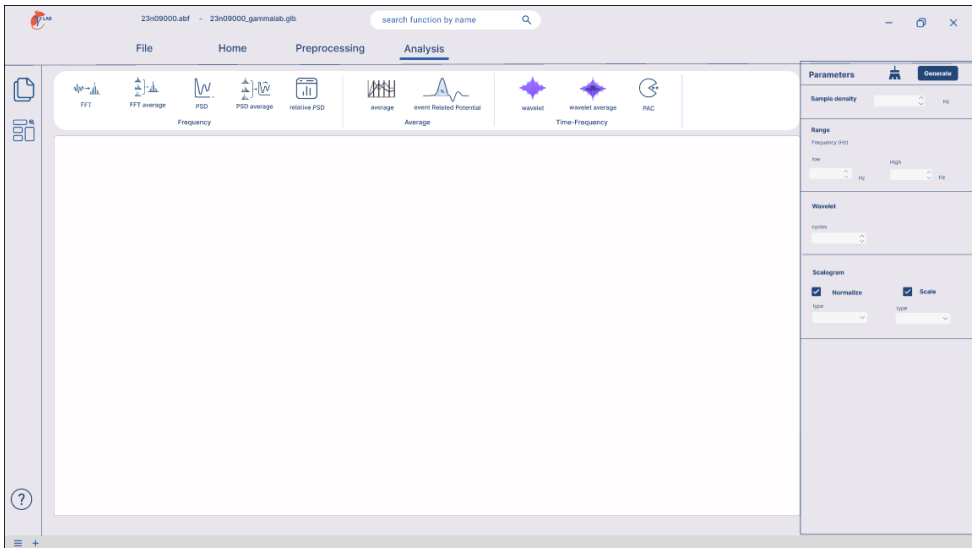


Figura 32: opciones de Wavelet

- **Wavelet Average:** calcula la transformada wavelet con la misma configuración que la opción anterior, y promedia los resultados sobre múltiples trials, generando un mapa tiempo-frecuencia agrupado.

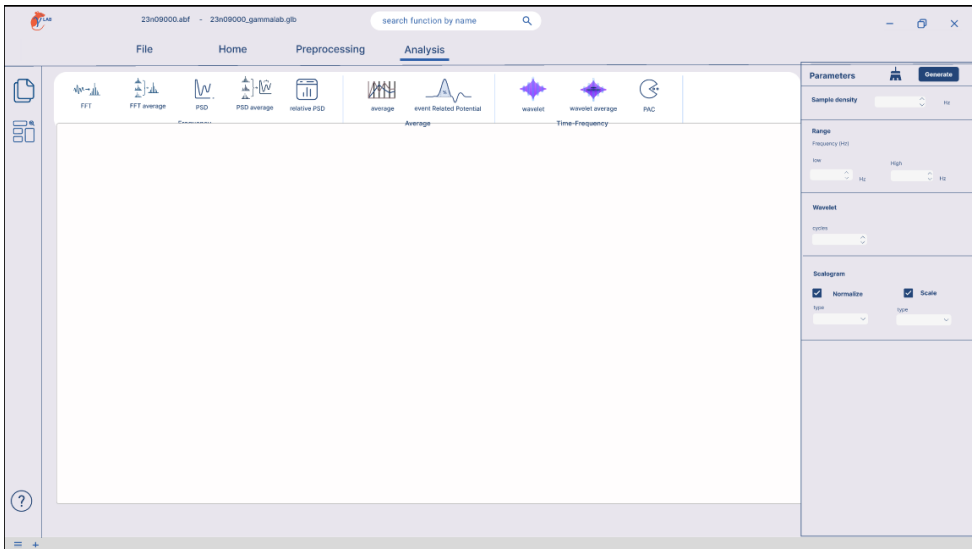


Figura 33: opciones de Wavelet Average

- **PAC:** calcula el acoplamiento de fase-amplitud (*Phase-Amplitude Coupling*) de un canal, evaluando la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias seleccionado, ya sea sobre un trial individual o sobre el promedio de varios trials. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de distribución de amplitud en función de la fase, y el vector de acoplamiento medio con su magnitud y ángulo.

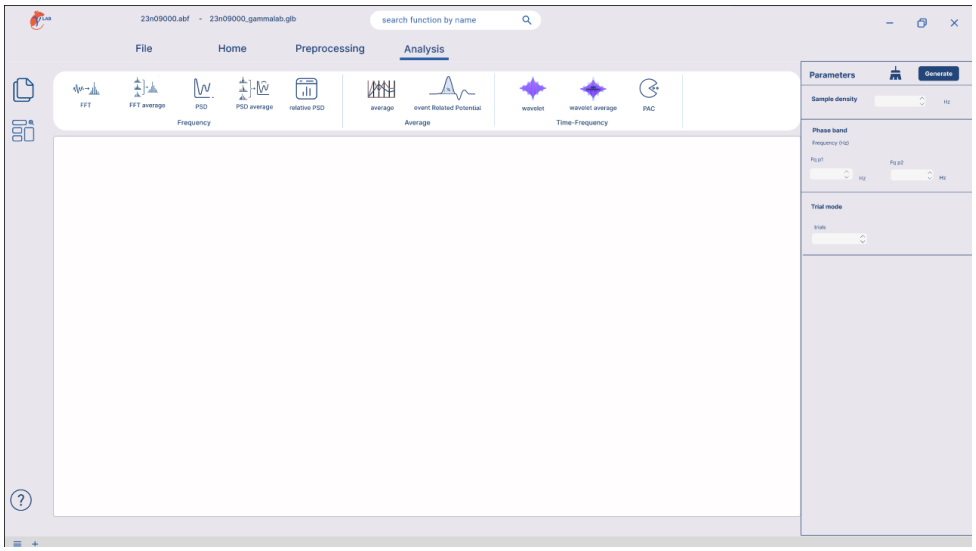


Figura 34: opciones de PAC